

Temas Selectos de Ingeniería Petrolera

31 de agosto de 2026

Moisés Velasco Lozano

PhD

PROFESOR

Universidad Nacional Autónoma de México

FACULTY RESEARCH GUEST

Los Alamos National Laboratory

Índice

1	Herramientas computacionales para el desarrollo científico	4
1.1	Python Orientado a Ciencia de Datos	4
1.1.1	Python	4
1.1.2	Manipulación y tratamiento de datos con NumPy y Pandas	6
1.1.3	Técnicas de visualización de datos usando Matplotlib y Seaborn	7
1.2	Gestión de Datos y Programación Colaborativa	9
1.2.1	Creación y administración de repositorios	10
1.2.2	Desarrollo de proyectos en entornos colaborativos	10

2 Herramientas computacionales para el desarrollo científico

COMPUTATIONAL TOOLS FOR SCIENTIFIC DEVELOPMENT

From Data to Insights • From Code to Collaboration

1. TOOLS FOR SCIENTIFIC COMPUTING

Powerful tools for data analysis, visualization, and modeling

Key Libraries

NumPy
Numerical computing

pandas
Data manipulation and analysis

Matplotlib
Data visualization and plotting

SciPy
Scientific computing

scikit-learn
Machine learning and statistics

xarray
Multi-dimensional array analysis

Applications in Geosciences

Data Analysis

Explore and clean geoscience data

Visualization

Create 2D/3D plots and maps

Geospatial Analysis

Work with spatial data and GIS formats

Modeling & Simulation

Build and analyze geoscience models

Machine Learning

Apply ML to predict and classify geoscience data

Example: 3D Visualization of Geological Data

```
import numpy as np
import xarray as xr
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Load data
data = xr.open_dataset('seismic.nc')
x, y, z = data.x, data.y, data.depth, data.amplitude

# 3D Visualization
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(x, y, z[50::10], cmap='seismic',
               sstride=5, cstride=5)
ax.set_xlabel('X (km)')
ax.set_ylabel('Y (km)')
ax.set_zlabel('Depth (m)')
plt.title('Seismic Amplitude (Depth Slice)')
plt.tight_layout()
plt.show()
```

Seismic Amplitude (Depth Slice)

2. DATA MANAGEMENT & COLLABORATIVE DEVELOPMENT WITH GITHUB

Organize data, track changes, and work together efficiently

Why GitHub?

Centralize and organize your data and code

Track changes with version control

Collaborate with others seamlessly

Backup, secure and reproducible workflows

Enable reproducible research

Collaborative Workflow

- Clone Repository**
Get a local copy of the project
- Create Branch**
Work on a new feature or fix
- Make Changes**
Edit code and analyze data
- Commit Changes**
Save your changes with a message
- Push to GitHub**
Upload your branch to the remote repo
- Open Pull Request**
Propose your changes for review
- Review & Merge**
Discuss, review and merge into main

Example Repository Structure

```
geoscience-project/
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
│   └── 01_exploratory_analysis.ipynb
├── scripts/
│   ├── data_cleaning.py
│   └── modeling.py
├── results/
│   └── figures/
├── README.md
└── environment.yml
```

Example: Working with Git

```
$ git clone https://github.com/user/geoscience-project.git
$ cd geoscience-project
$ git checkout -b feature/data-analysis

# make changes...
$ git add .
$ git commit -m "Add data analysis notebook"
$ git push origin feature/data-analysis

# open Pull Request on GitHub
# after review and approval
# your changes are merged!
```

Better Analysis
Better Decisions

Reproducible
Research

Stronger Collaboration
Greater Impact

Open Science
Open Future

1 Herramientas computacionales para el desarrollo científico

El desarrollo científico en geociencias depende cada vez más del uso de herramientas computacionales capaces de facilitar el procesamiento, análisis, visualización y modelado de grandes volúmenes de información. En áreas como la ingeniería petrolera, los datos pueden provenir de fuentes muy diversas, incluyendo registros de pozos, análisis petrofísicos, información sísmica, modelos geológicos, simulaciones numéricas, datos experimentales y mediciones de producción. La correcta integración y análisis de esta información constituye una parte fundamental del proceso de investigación y de la toma de decisiones.

Dentro del ecosistema de programación científica, Python se ha consolidado como uno de los lenguajes más utilizados debido a su sintaxis accesible, flexibilidad y amplio conjunto de librerías especializadas. Adicionalmente los sistemas de control de versiones como Git, junto con plataformas como GitHub, proporcionan un entorno para organizar y documentar el desarrollo de proyectos computacionales. Su utilización permite mantener un historial de modificaciones, recuperar versiones anteriores, desarrollar nuevas funcionalidades de manera independiente e integrar posteriormente las contribuciones realizadas por diferentes participantes.

1.1 Python Orientado a Ciencia de Datos

Objetivo específico: Comprender los fundamentos de Python y su importancia como lenguaje líder para el desarrollo de aplicaciones de Ciencia de Datos e Inteligencia Artificial.

1.1.1 Python

Python es un lenguaje de programación interpretado, de alto nivel, multiparadigma y de código abierto. Su sintaxis sencilla y su amplio ecosistema de bibliotecas lo han convertido en el lenguaje predominante para el análisis de datos, la inteligencia artificial y el aprendizaje automático. Entre sus principales ventajas destacan:

- Sintaxis clara y legible.
- Gran comunidad de desarrolladores.
- Amplio conjunto de bibliotecas científicas.
- Compatibilidad con múltiples sistemas operativos.
- Integración con herramientas de Big Data y computación en la nube.

Una de las principales ventajas de Python para el desarrollo científico es su amplio ecosistema de bibliotecas especializadas (**Figura 1**), las cuales permiten implementar tareas complejas sin necesidad de desarrollar todas las herramientas desde cero. Bibliotecas como NumPy facilitan el cálculo numérico y manejo de arreglos multidimensionales; pandas permite organizar, procesar y analizar conjuntos de datos; Matplotlib proporciona herramientas para su visualización; y SciPy incorpora métodos para optimización, integración, interpolación y análisis estadístico. Adicionalmente, bibliotecas como scikit-learn, TensorFlow y PyTorch permiten implementar algoritmos de Machine Learning y Deep Learning.

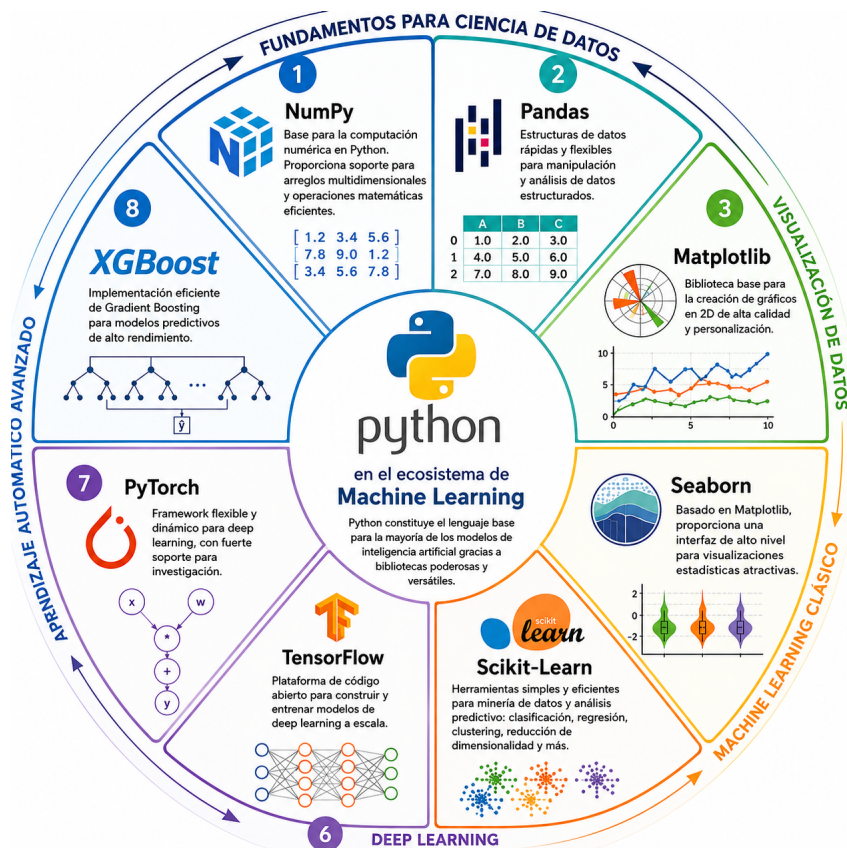


Figura 1: Librerías en Python para el análisis de datos y desarrollo de modelos de machine learning

Estas herramientas permiten cubrir todo el ciclo de vida de un proyecto de Machine Learning (**Figura 2**):

1. Obtención de datos.
2. Limpieza y preparación.
3. Exploración.
4. Entrenamiento del modelo.
5. Evaluación.
6. Implementación.

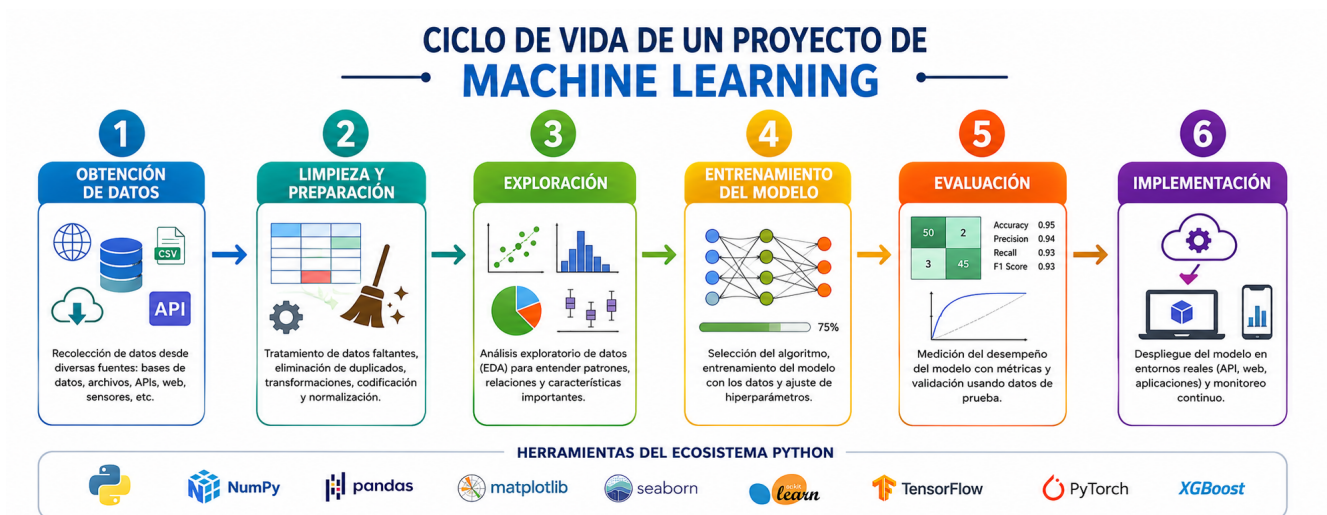


Figura 2: Etapas de desarrollo de un proyecto de machine learning

1.1.2 Manipulación y tratamiento de datos con NumPy y Pandas

Objetivo específico. Aplicar técnicas de manipulación de datos utilizando las bibliotecas NumPy y Pandas (**Figura**) para preparar conjuntos de datos destinados al entrenamiento de modelos de Machine Learning.

En cualquier proyecto de Ciencia de Datos, aproximadamente entre el 70% y el 80% del tiempo se dedica a la preparación de los datos. Las bibliotecas/librerías más utilizadas son:

1. NumPy

NumPy es una biblioteca fundamental de Python para el cálculo numérico y científico. Permite trabajar eficientemente con arreglos multidimensionales y realizar operaciones matemáticas, estadísticas, matriciales y algebraicas, siendo la base de muchas otras bibliotecas utilizadas en ciencia de datos, machine learning e ingeniería.

2. Pandas

Pandas es una biblioteca de Python diseñada para la manipulación, organización y análisis de datos. Utiliza estructuras como DataFrame y Series, que permiten importar, limpiar, filtrar, transformar y combinar grandes conjuntos de información de manera eficiente.

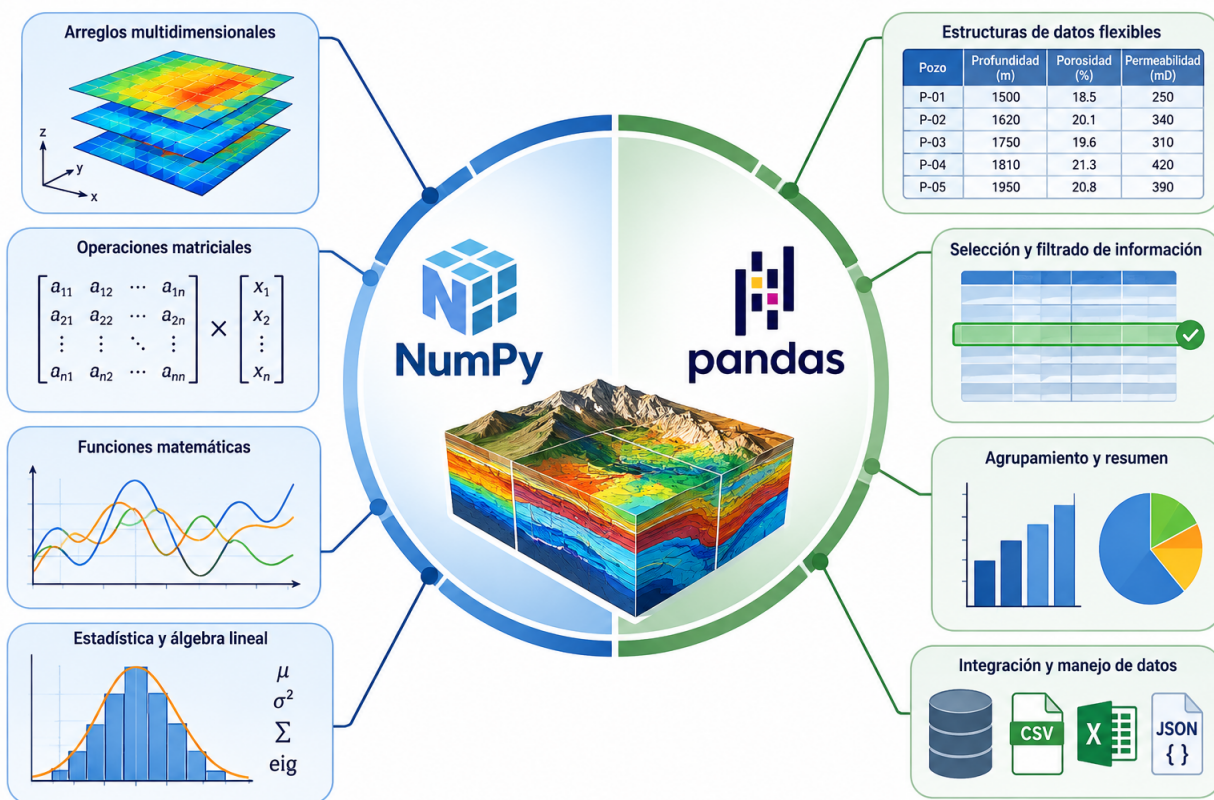


Figura 3: Librerías NumPy y Pandas

1.1.3 Técnicas de visualización de datos usando Matplotlib y Seaborn

Objetivo específico. Revisar las librerías Matplotlib y Seaborn, las cuales permiten representar gráficamente la información, facilitar la exploración de datos y apoyar en la toma de decisiones durante el desarrollo de análisis de datos.

La visualización permite:

1. Detectar valores atípicos.
2. Identificar correlaciones.
3. Comprender distribuciones.
4. Descubrir patrones ocultos.
5. Comunicar resultados.

Existen diversas librerías especializadas para la visualización de datos y resultados, entre las cuales se tienen:

1. Matplotlib

Matplotlib es una biblioteca de Python utilizada para la visualización de datos mediante la creación de gráficos y figuras. Permite representar información de forma clara a través de gráficas de líneas, dispersión, barras, histogramas, mapas de calor y otros tipos de visualizaciones. Es ampliamente utilizada en ciencia, ingeniería y geociencias para explorar datos, identificar tendencias y comunicar resultados.

2. Seaborn

Seaborn es una biblioteca de Python especializada en la visualización estadística de datos. Está construida sobre Matplotlib y facilita la creación de gráficos atractivos y útiles para explorar distribuciones, relaciones entre variables, correlaciones y patrones presentes en los datos.

Estas librerías (**Figura 4**) tienen funcionalidades que permiten explicar de manera eficiente el comportamiento de los datos, tendencias, y modelos dinámicos.

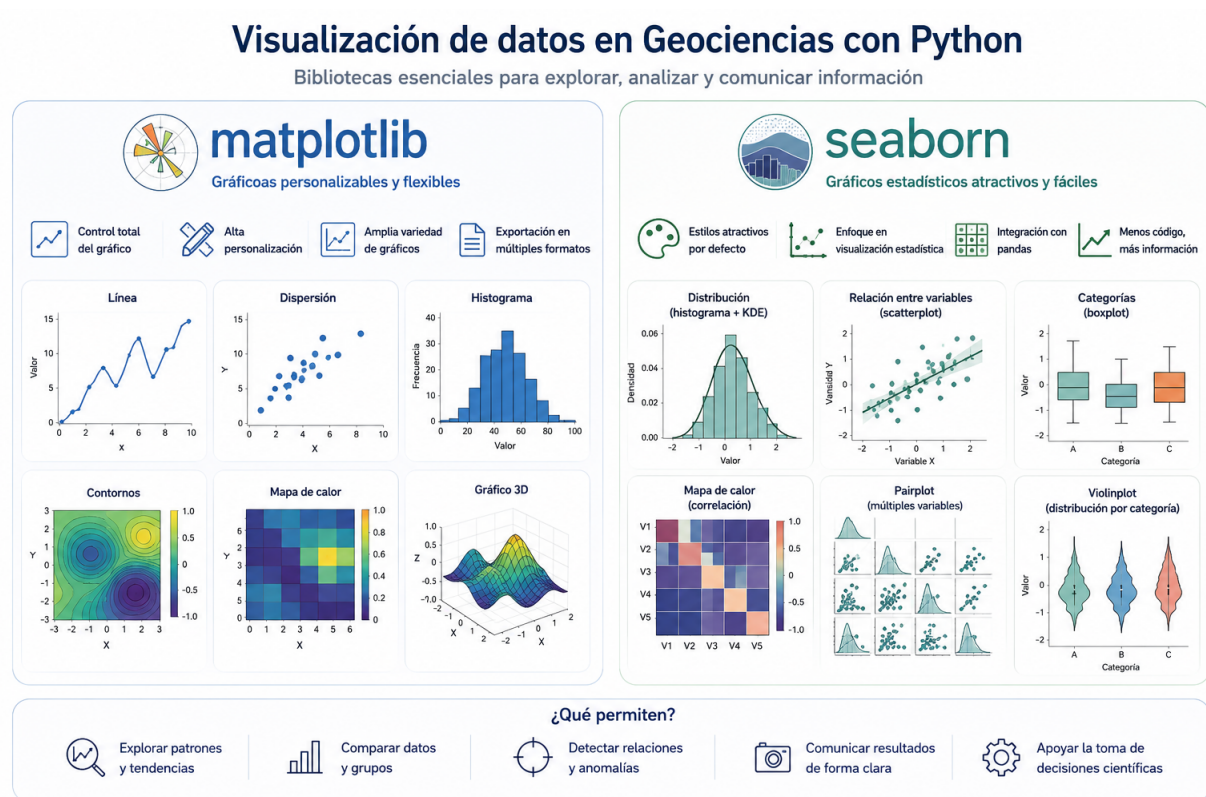


Figura 4: Librerías Matplotlib y Seaborn

1.2 Gestión de Datos y Programación Colaborativa

Objetivo específico. Comprender los fundamentos y utilización de GitHub para administrar el ciclo de vida de proyectos de ciencia de datos aplicados a geociencias.

La utilización de repositorios de información es fundamental en el desarrollo de modelos basados en datos. Adicionalmente, el control de versiones y uso de plataformas colaborativas permiten gestionar proyectos aplicando buenas prácticas en el desarrollo de software, la administración de base de datos y la colaboración en equipos multidisciplinarios de investigación.

Los proyectos de machine learning suelen involucrar múltiples archivos, modelos, bases de datos, experimentos y colaboradores. Sin una adecuada gestión del código fuente, resulta difícil controlar cambios, reproducir experimentos o trabajar en equipo.

El control de versiones permite registrar la evolución de un proyecto, recuperar versiones anteriores y coordinar el trabajo entre varios desarrolladores sin sobrescribir cambios. En este contexto, Git es el sistema de control de versiones distribuido más utilizado en el mundo, mientras que GitHub constituye una plataforma basada en la nube que facilita el almacenamiento, la colaboración y la gestión integral de proyectos de software.

GitHub (Figura) es una plataforma de alojamiento de repositorios Git que incorpora herramientas para la colaboración, documentación, automatización e integración continua. Entre sus funcionalidades destacan:

- Repositorios públicos y privados.
- Gestión de proyectos

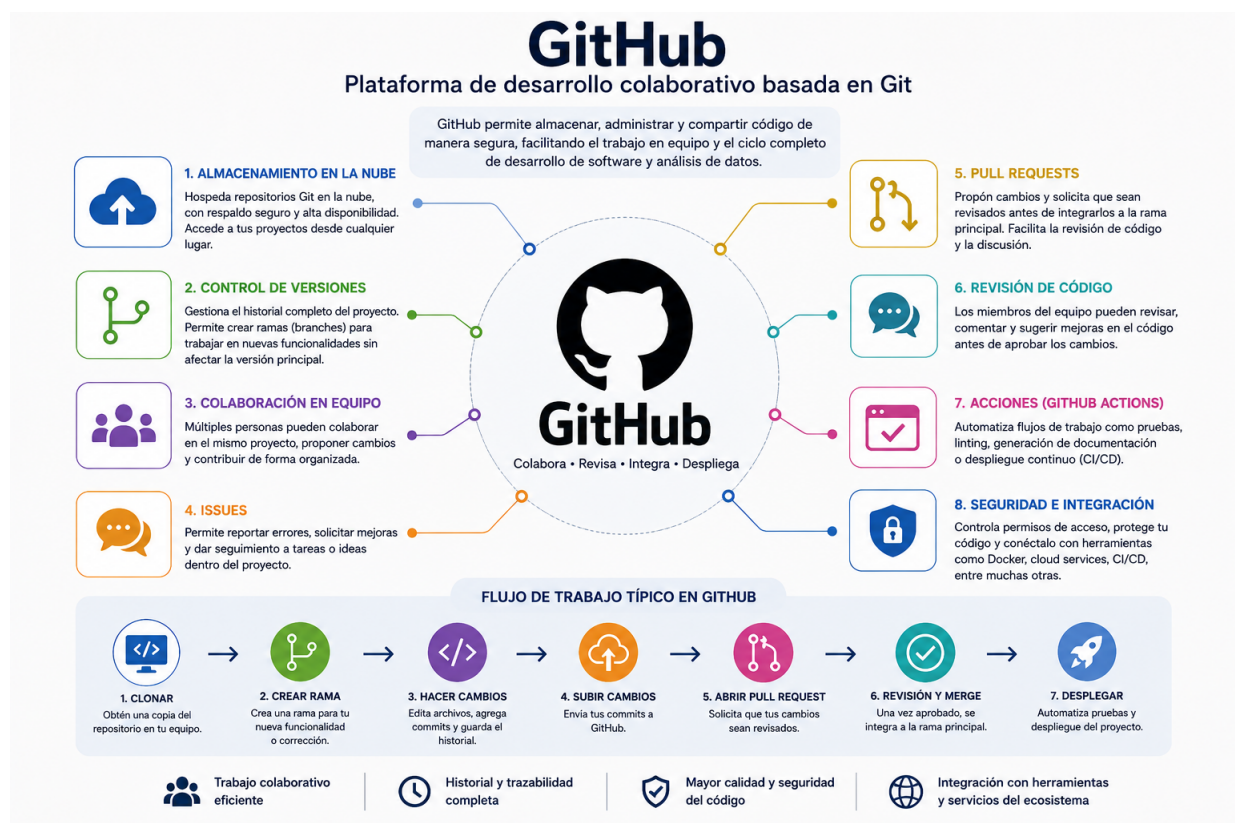


Figura 5: Funcionalidades de la herramienta GitHub

GitHub se ha convertido en un estándar para proyectos de software, investigación científica y desarrollo de inteligencia artificial.

1.2.1 Creación y administración de repositorios

Objetivo específico. Diseñar y administrar repositorios que faciliten la organización, documentación, reproducibilidad y difusión de proyectos de investigación en Ciencia de Datos y Machine Learning.

La reproducibilidad es uno de los pilares de la investigación científica. Un repositorio bien organizado permite que otros investigadores comprendan, ejecuten y validen los resultados obtenidos. GitHub se ha consolidado como una plataforma para compartir código, datos y documentación, promoviendo la ciencia abierta y el trabajo colaborativo. La (Figura) ilustra la principales funcionalidades de GitHub como herramienta de almacenamiento de información.

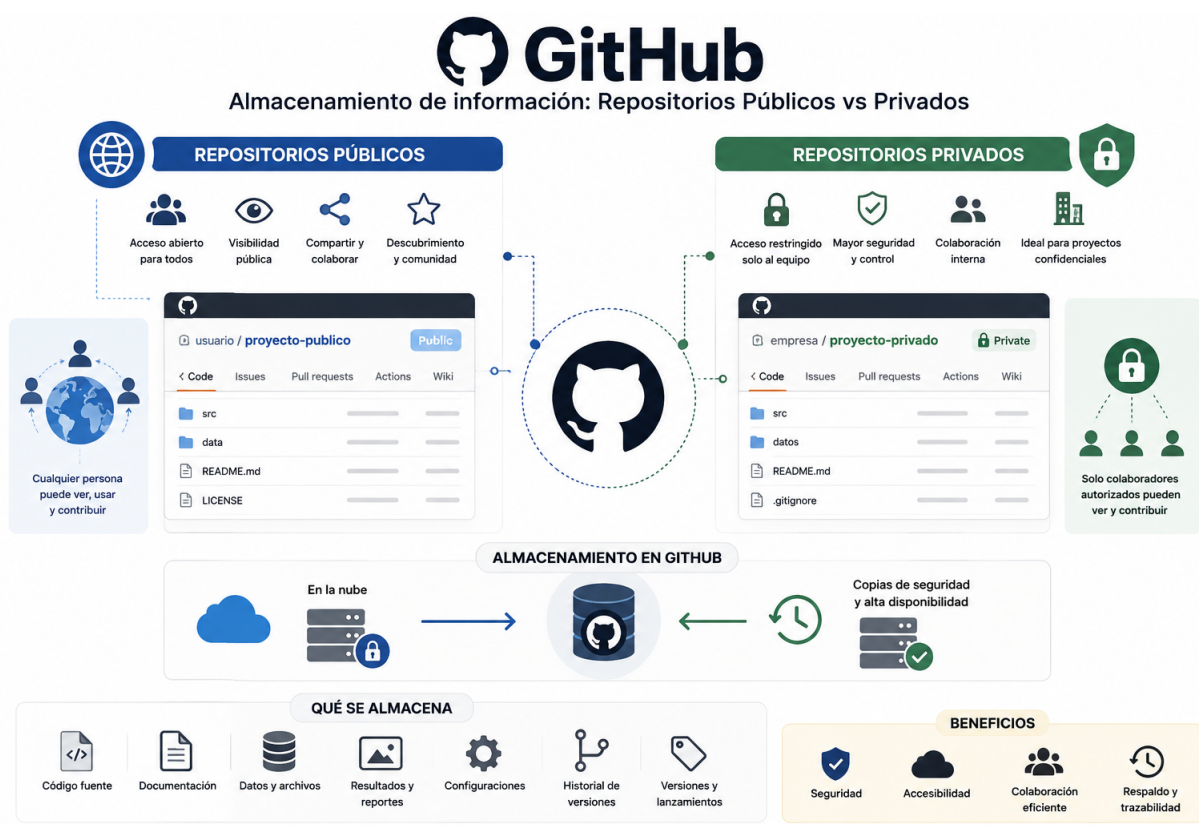


Figura 6: GitHub como repositorio de información

1.2.2 Desarrollo de proyectos en entornos colaborativos

Objetivo específico. Aplicar metodologías y herramientas colaborativas para desarrollar proyectos de manera organizada, eficiente y reproducible.

Los proyectos actuales de Machine Learning rara vez son desarrollados por una sola persona. Generalmente participan:

Científicos de datos	Ingenieros de software	Especialistas del dominio	Analistas de negocio	Administradores de bases de datos
----------------------	------------------------	---------------------------	----------------------	-----------------------------------

El desarrollo colaborativo consiste en que múltiples desarrolladores trabajan simultáneamente sobre un mismo proyecto sin generar conflictos importantes. Principios fundamentales:

Comunicación	Desarrollo de versiones	Documentación	Automatización	Revisión entre pares
--------------	-------------------------	---------------	----------------	----------------------

La (Figura) muestra algunas de las funcionalidades de GitHub para el desarrollo de proyectos colaborativos.

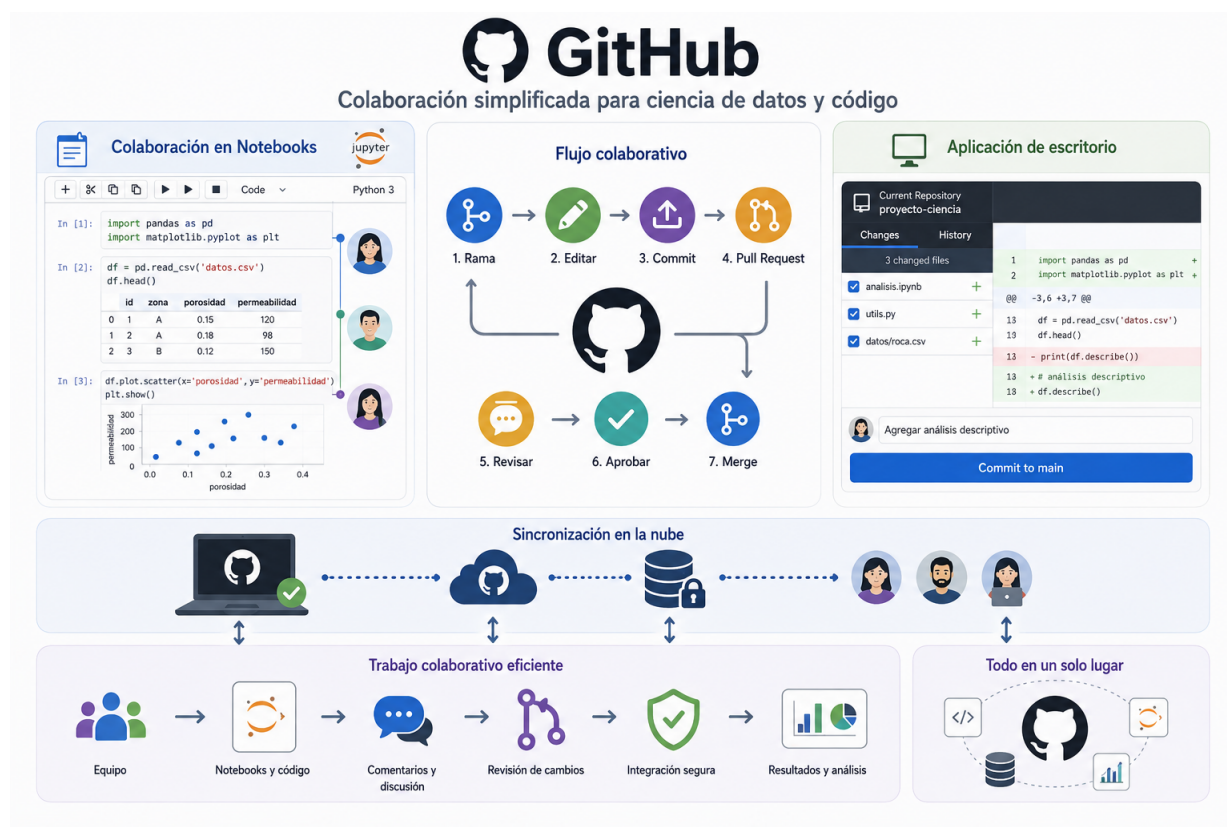


Figura 7: GitHub para desarrollo colaborativo

Flujo de trabajo colaborativo. Un flujo típico comprende (Figura):



Figura 8: Flujo de trabajo colaborativo